```
1  //Example of Parameterised Constructor
2
3  /*
4  A constructor that has parameters is known as parameterised
5  constructor. If we want to initialize fields of the class with
6  our own values, then use a parameterised constructor.
7   */
8
9  import java.util.Scanner;
10 class Test
11 {
12    int num;
13
14    //Parameterised Constructor of class Test with one parameter
15    Test(int num)
16    {
17       System.out.println("The value of instance num before
   initialisation by constructor: "+this.num);
18
19       /*
20       Assigning value to a instance or global variable 'num' of class Test,
21       by local variable 'num' which has value from user input.
22        */
23       this.num=num;
24       /*
25       this keyword refers to the current object in a method or constructor
26        */
27
28    }
29    void disp()
30    {
31       System.out.println("The value of instance num after
   initialisation from constructor: "+num);
32    }
33 }
34
35 public class Main {
36    public static void main(String[] args) {
37       Scanner sc=new Scanner(System.in);
38       System.out.println("Input a number to initialize the instance
   variable num of class Test: ");
39
40       //public int nextInt() defined in class Scanner
41       int number=sc.nextInt();
```

```
42
43          Test obj=new Test(number);
44          //Will invoke parameterised constructor
45
46          /*
47          Test obj=new Test();
48          if you write this statement it will raise an error because when you
49          create a constructor of any type by own, compiler will not create
50          a default one by its own. Here, as we have created a parameterised
51          constructor, and not created a no-argument constructor it has
52          neither no argument (by programmer) nor default constructor (by
53          compiler). It has only parameterised which is Test(int num), so for
54          calling this, we have to pass a int range value in the parenthesis
55          while object/instance creation of same class.
56           */
57
58          obj.disp();
59      }
60  }
```